



Sistemi Distribuiti

Laurea in Informatica

Michele Ciavotta

michele.ciavotta@unimib.it

XML - HTML - JSON

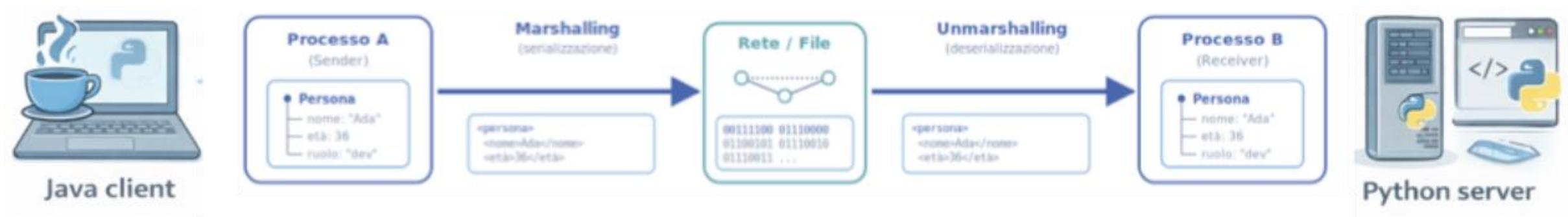
FORMATO DEI DATI

Contenuto della lezione

- Cos'è un linguaggio di markup basato su <tag>
- HTML (+ DOM + CSS)
- XML (+ DTD, XML Schema)
- JSON

FORMATI DI SCAMBIO

Perché i formati di scambio sono importanti?



Il problema: sistemi che non si conoscono devono comunicare

- I componenti di un sistema distribuito sono sviluppati **indipendentemente**, in linguaggi e su piattaforme diverse
- Non esiste una **memoria condivisa**: i dati viaggiano sulla rete come sequenze di byte
- Serve un accordo su **come interpretare quei byte**: il formato dei dati è quel contratto

Interoperabilità

- Un client Java deve poter parlare con un server Python, un database Oracle, un browser
- **XML, JSON, HTML** sono formati **indipendenti dal linguaggio e dalla piattaforma**
- Il mittente **serializza**, il destinatario **deserializza** — entrambi conoscono il formato

Perché i formati di scambio sono importanti?



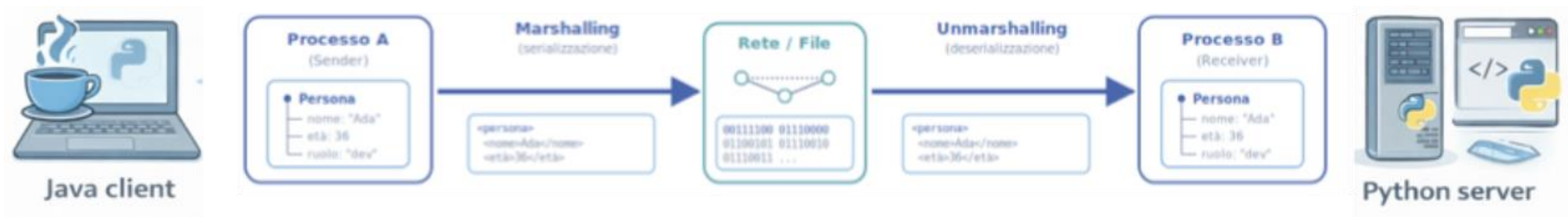
Marshalling (serializzazione)

- Conversione di strutture dati in memoria in un formato trasmissibile (testo o byte)
- Necessario per salvare su file, inviare via rete o passare tra processi

Unmarshalling (deserializzazione)

- Ricostruzione delle strutture dati originali a partire dal formato serializzato
- Il ricevente deve conoscere lo schema/formato per interpretare correttamente i dati

Perché i formati di scambio sono importanti?



Validazione e contratti strutturali

- DTD e XML Schema permettono di definire e verificare la **struttura attesa** dei dati
- Un messaggio malformato può essere rifiutato **prima** di entrare nella logica applicativa
- JSON è più agile ma richiede **disciplina** nei sistemi grandi (→ JSON Schema)

Disaccoppiamento e leggibilità

- Formati testuali (XML, JSON) sono **leggibili da umani e da macchine**: facilitano debug, logging e monitoraggio
- Separano la **struttura dei dati** dalla logica che li processa: i componenti evolvono indipendentemente
- Formati binari (Protocol Buffers, BSON, Avro): compatti e veloci da processare
- Il testo favorisce interoperabilità, portabilità e versionamento (diff/merge con VCS)
- Il binario è preferito quando servono alte prestazioni o volumi elevati di dati

Introduzione ai linguaggi di markup

Cosa sono

- Sistemi di annotazione del testo tramite tag che descrivono struttura
- e semantica dei dati; distinti dai linguaggi di programmazione

Cenni storici

- Anni '60: IBM sviluppa **GML** (Goldfarb, Mosher, Lorie) per la gestione documentale
- 1986: GML evolve nello standard ISO **SGML** (meta-linguaggio per definire markup)
- 1991: Berners-Lee crea **HTML** (sottoinsieme semplificato di SGML per il Web)
- 1998: il W3C pubblica **XML** (sottoinsieme rigoroso di SGML, estensibile e general-purpose)

Obiettivi principali

- Separazione tra contenuto e presentazione
- Strutturazione gerarchica dell'informazione
- Interoperabilità tra sistemi e portabilità dei dati
- Leggibilità sia per l'uomo che per la macchina

LINGUAGGI DI MARKUP

Linguaggi di markup

Linguaggio utilizzato per **comunicare istruzioni a una macchina di calcolo**, per definire programmi che controllino il comportamento di un calcolatore

Linguaggio di markup: utilizzato per **annotare un documento** in modo tale che l'annotazione sia **sintatticamente distinguibile** dal testo

Le annotazioni possono avere diverse finalità:

- **descrittive** (etichettano semplicemente parti del testo, disaccoppiando la struttura dalla presentazione del testo stesso)
- **procedurali** (definiscono istruzioni per programmi che elaborino il testo al quale sono associate)
- di **presentazione** (definiscono come visualizzare il testo al quale sono associate)

Rappresentazione delle informazioni

Serve un formalismo *flessibile per*

- la separazione tra:
 - *contenuto*
 - *presentazione*
 - *navigazione*
- la definizione di *domini* o *contesti*
- l'indipendenza dalla piattaforma (*media*) e supporto multilingue

Risposta: linguaggi di marcatura dei documenti

- *Marcatura* (etichettatura o tag) per rendere esplicita un'interpretazione di un testo
- *Marcatura (tag)*: qualsiasi tipo di codice inserito in un testo in forma elettronica

Si distinguono due tipi di marcatura:

- *marcatura procedurale*
 - descrive come processare il documento (PostScript, PDF, RTF, formato MS Word)
- *marcatura descrittiva*
 - descrive la struttura logica del (HTML, SGML, XML)

Contenuti parzialmente derivati da: <http://www.diag.uniroma1.it/~rosati/ltw-1819/>

- Minima introduzione su HTML
- Minima introduzione su CSS
- Minima introduzione su DOM

HTML

HTML: Introduzione

- Cos'è l'HTML?
- HTML (Hyper Text Markup Language) è un linguaggio di markup per dare struttura ai contenuti (web)
- La specifica HTML è definita dal W3C (World Wide Web Consortium)

Dal web degli autori di pagine HTML scritte a mano...



... al web delle pagine HTML generate da programmi (sistemi di gestione di blog o CMS)

<http://visual.ly/html5-past-present-and-future>

HTML: a che serve?

whichElement?

Trying to answer that age old question:

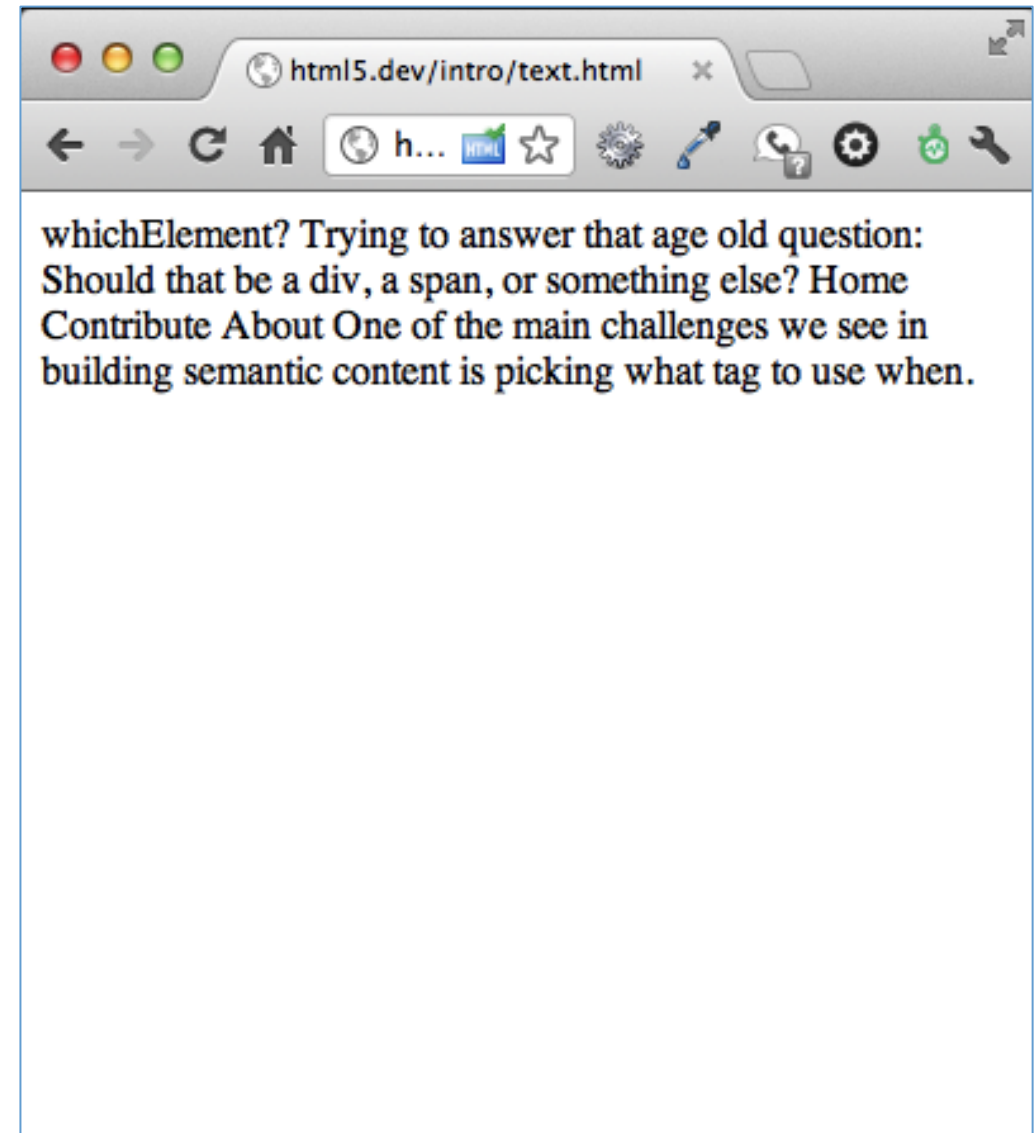
Should that be a div, a span, or something else?

Home

Contribute

About

One of the main challenges we see in building semantic content is picking what tag to use when.



HTML: tag semantici pre-HTML5

Questi tag esistono da **HTML 1/2/4** e sono sempre stati semantici — molto prima che HTML5 introducesse `<nav>`, `<section>` ecc.

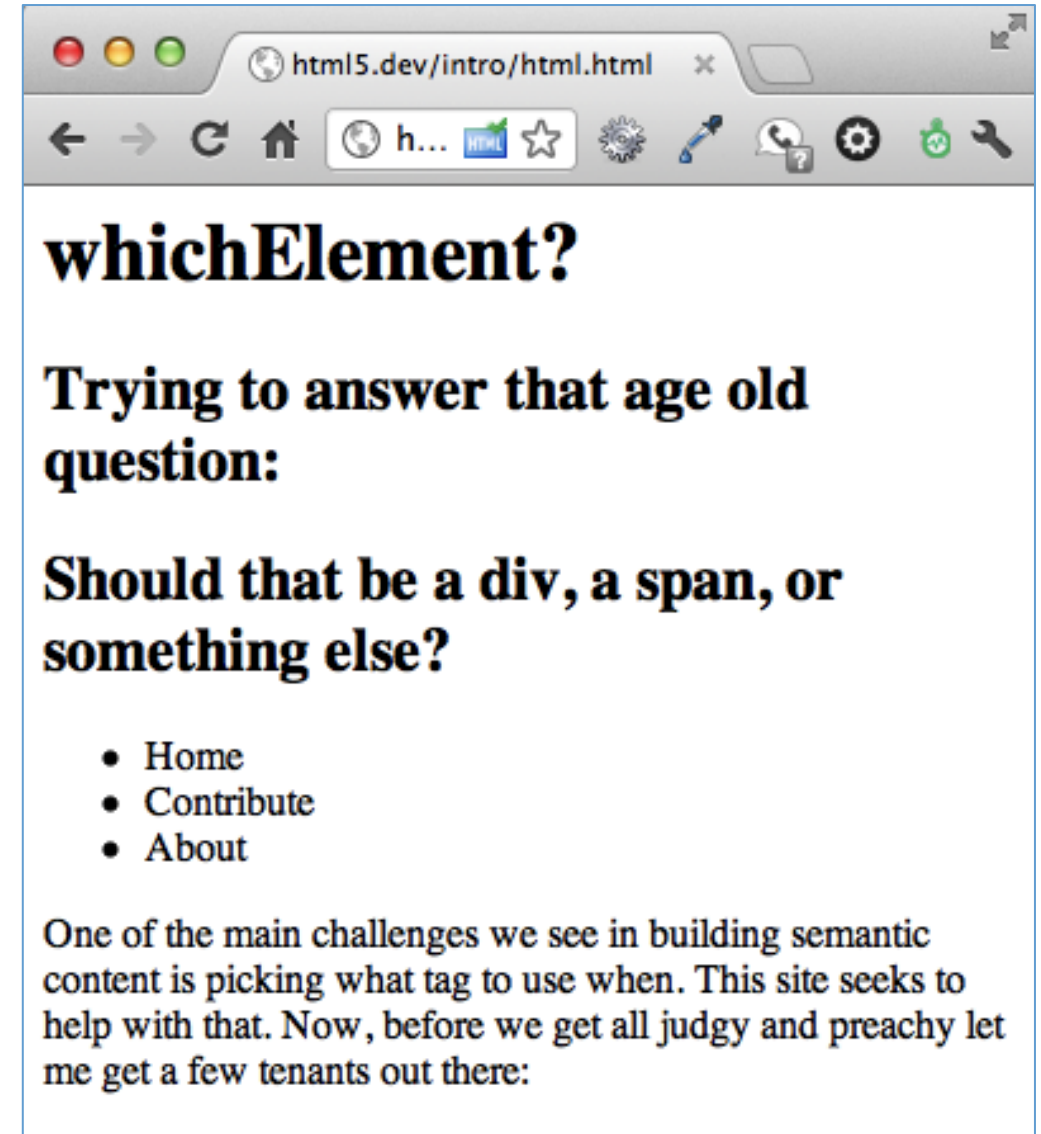
Tag	Significato
<code><h1>–<h6></code>	Intestazioni gerarchiche
<code><p></code>	Paragrafo
<code></code> , <code></code> , <code></code>	Liste (non ordinata, ordinata, elemento)
<code><table></code>	Dati tabulari
<code><form></code> , <code><input></code>	Form interattivi
<code><a></code>	Collegamento ipertestuale
<code></code> , <code></code>	Enfasi semantica

`<em>` e `<strong>` sono semantici (significano *enfasi* e **importanza**), a differenza di `<i>` e `<b>` che sono solo visivi (corsivo e grassetto senza significato).

HTML: Rendering base

```
<h1>whichElement?</h1>
<h2>Trying to answer that age old question:</h2>
<h2>Should that be a div, a span, or something else?</h2>
<ul>
  <li>Home</li>
  <li>Contribute</li>
  <li>About</li>
</ul>
<p>One of the main challenges we see in building semantic
content is picking what tag to use when. This site seeks
to help with that. Now, before we get all judgy and
preachy let me get a few tenants out there:</p>
```

Esempio da: <http://www.slideshare.net/tpryan/introduction-to-html5-11711719>



HTML: I form

Quando si richiedono dei dati a un utente (es. informazioni di registrazione) serve poter includere in una pagina dei controlli tipici di interfacce utenti tradizionali, quali:

- Caselle di testo
- Checkbox/radio button
- Select / combo box
- Selettori di file, date, ...
- Bottoni
- Altri ancora

Questi controlli hanno dei tag HTML dedicati, ad es.:

```
<label for="fname">First name:</label><br>  
<input type="text" id="fname" name="fname" value="John"><br>  
<label for="lname">Last name:</label><br>  
<input type="text" id="lname" name="lname" value="Doe"><br><br>  
<input type="submit" value="Submit">
```

Example

First name:

Last name:

[Try it Yourself »](#)

HTML: I form

- Il form stesso è associato a un tag specifico:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>HTML Forms</h2>
```

```
<form action="/action_page.php">
```

```
  <label for="fname">First name:</label><br>
```

```
  <input type="text" id="fname" name="fname" value="John"><br>
```

```
  <label for="lname">Last name:</label><br>
```

```
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

```
<p>If you click the "Submit" button, the form-data will be sent to a page called  
"/action_page.php".</p>
```

```
</body>
```

```
</html>
```

- Viene indicata tramite attributo **action** almeno una URL riferita allo script lato server al quale vanno passati i dati
- Un attributo opzionale molto rilevante è **method** (GET o POST), altri attributi sono **target** («dove» visualizzare i risultati dello script indicato in action) e **enctype** (eventuale codifica crittografica dei dati prima dell'invio)

Form con GET

L'attributo *method* determina anche il modo con cui vengono passati i parametri

- La GET include eventuali parametri in coda alla URL della risorsa (detti *query*)

```
GET resource[?key=value{&key=value}] HTTP1.1  
{header lines}
```

- Esempio:

```
1. <form action="myCompany/orders" method="GET">  
2.   Product: <input type="text" id="item" name="item">  
3.   Date: <input type="text" id="date" name="date">  
4.   <input type="submit" value="Get orders">  
5. </form>
```

dammi tutti gli ordini di marzo per il prodotto con codice Q2345

```
GET myCompany/orders?item="Q2345"&date="2022/03" HTTP1.1
```

Form con POST

- La POST prevede che i dati vengano messi in coda come documento autonomo (detto *body*)

POST *resource* HTTP1.1

{header lines}

body

- Esempio:

```
1.<form action="myCompany/orders" method="POST">
2.  <input type="hidden" id="update" value="true">
3.  Old item: <input type="text" id="oldItem" name="oldItem">
4.  New item: <input type="text" id="newItem" name="newItem">
5.  Date: <input type="text" id="date" name="date">
6.  <input type="submit" value="Update orders">
7.</form>
```

aggiorna un codice prodotto in tutti gli ordini di aprile

POST *myCompany/orders* HTTP1.1

Content-Length: 59

Content-Type: application/x-www-form-urlencoded

update=true&oldItem="Q2345"&newItem="Q68254"&date="2022/04"

Cosa c'è di nuovo in HTML5?

Nuovi **elementi semantici** per la strutturazione delle pagine

- article, section, aside, header, footer, etc

Nuovi **elementi di input e multimediali**

- Widget per input search, email, url, number, tel, ma anche range, date...
 - ... purtroppo supporto non uniforme da tutti i browser
- audio, video, canvas

Nuove **API JavaScript** per la manipolazione delle pagine

- Offline data, drag and drop, web storage

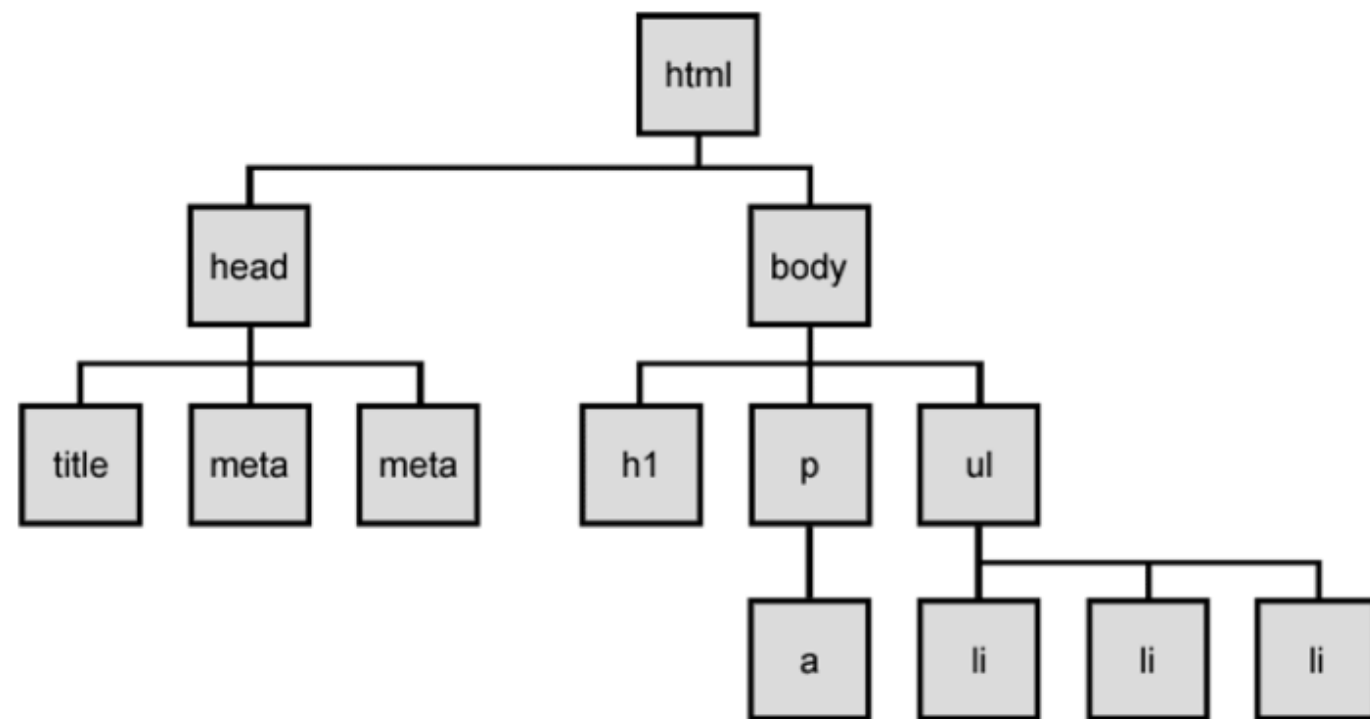
DOM: struttura ad albero

Cosa si intende per DOM?

DOM (Document Object Model) è una **interfaccia neutrale** rispetto al linguaggio di programmazione e alla piattaforma utilizzata, per consentire ai programmi **l'accesso e la modifica dinamica** di contenuto, struttura e stile di un documento web.

È una **API** definita dal **W3C** e implementata da ogni browser moderno per l'accesso e la gestione dinamica di documenti **XML** e **HTML**.

DOM: navigazione dei nodi



```
<html>
  <head>
    <title>Example ...</title>
    <meta charset="UTF-8">
    <meta name="author" content="John Doe">
  </head>
  <body>
    <h1>Heading 1</h1>
    <p>This is a paragraph of text with a
      <a href="/path/page.html">link in it</a>.
    </p>
    <ul>
      <li>Coffee</li>
      <li>Tea</li>
      <li>Milk</li>
    </ul>
  </body>
</html>
```

DOM: identificatori e ricerca nodi

Ogni nodo può essere caratterizzato da attributi che ne facilitano l'identificazione, la ricerca, la selezione

- Un identificatore **univoco** (attenzione: il DOM non garantisce l'unicità)
- Una "classe" che indica l'appartenenza ad un insieme che ci è utile definire

Il browser stesso fornisce funzionalità di ricerca:

- `getElementById(IdName)`
- `getElementsByClassName(Classname)`
- ... diverse altre meno interessanti

```
<h1>A One Page FAQ</h1>
<div id="introText">
  <p>bla bla bla</p>
</div>
<h2>I've heard that JavaScript is the long-lost fountain of youth. Is this true?</h2>
<div class="answer">
  <p>bla bla bla</p>
</div>
<h2>Can JavaScript really solve all of my problems?</h2>
<div class="answer">
  <bla bla bla</p> ← tag di apertura non valido: manca "p" dopo "<"
</div>
```

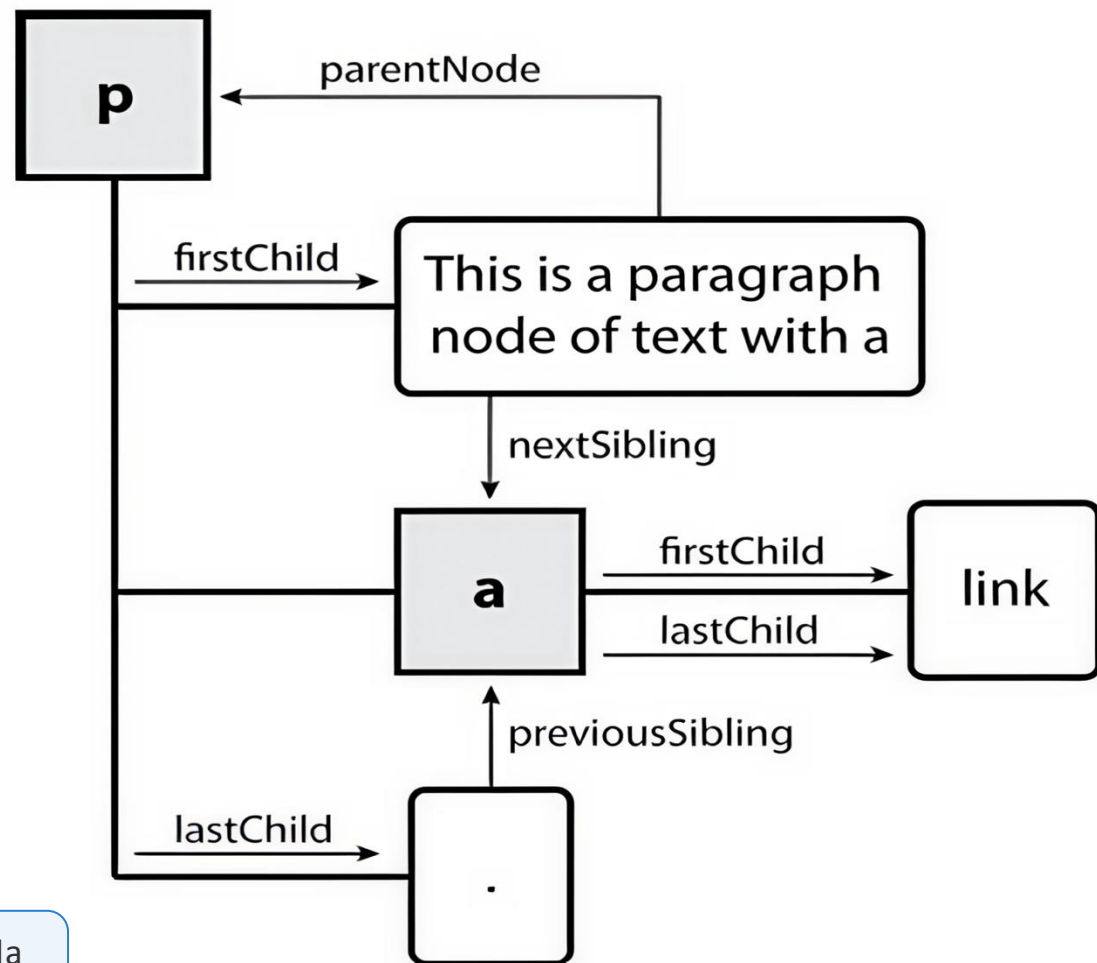

DOM: manipolazione

```
<p id="foo">
  This is a paragraph of text with a
  <a href="/path/to/page.html">link</a>
  .
</p>
```

Proprietà DOM	Significato
<code>parentNode</code>	Il nodo genitore diretto
<code>childNodes</code>	Lista di tutti i figli
<code>firstChild</code>	Primo figlio del nodo
<code>lastChild</code>	Ultimo figlio del nodo
<code>nextSibling</code>	Nodo fratello successivo
<code>previousSibling</code>	Nodo fratello precedente

Con questa struttura **JavaScript può raggiungere e modificare qualsiasi elemento** della pagina in modo dinamico.

Il browser converte l'HTML in un **albero di oggetti**. Ogni tag HTML diventa un **nodo**: padre, figlio o fratello degli altri.



DOM: eventi

```
// seleziono un elemento
const btn = document.getElementById("myBtn");

// aggiungo un listener
btn.addEventListener("click", function(e) {
  console.log("cliccato!", e.target);
});
```

Un **evento** è un segnale che qualcosa è accaduto nel documento (click, tasto, caricamento...). JavaScript può **ascoltarlo e reagire** tramite un listener.

Categoria	Eventi principali
Mouse	click, dblclick, mouseover, mouseout
Tastiera	keydown, keyup, keypress
Form	submit, change, focus, blur, input
Documento	DOMContentLoaded, load, resize, scroll
Touch	touchstart, touchend, touchmove

CSS: cos'è e sintassi base

Definizione

- CSS (**Cascading Style Sheets**) = linguaggio per definire lo **stile di presentazione** dei contenuti web
- Specifica mantenuta dal **W3C**; versione attuale: CSS3 (modulare)
- Realizza la **separazione contenuto/presentazione**: HTML struttura, CSS veste

Il concetto di cascata

- Più fogli di stile possono sovrapporsi; i conflitti si risolvono per:
 - **specificità** del selettore (id > classe > elemento)
 - **ordine di dichiarazione** (l'ultima regola prevale a parità di specificità)

Sintassi di una regola CSS

- **selettore { proprietà: valore; }**
- Es.: `h1 { color: navy; font-size: 2em; }`

Modi di inclusione

- **Inline**: attributo `style=""` sul tag (sconsigliato)
- **Interno**: blocco `<style>` nel `<head>`
- **Esterno** (best practice): file `.css` collegato con `<link rel="stylesheet">`

CSS: selettori

Selettori fondamentali

- **Elemento:** `p { ... }` — seleziona tutti i `<p>`
- **Classe:** `.info { ... }` — elementi con `class="info"`
- **ID:** `#header { ... }` — elemento unico con `id="header"`
- **Universale:** `* { ... }` — tutti gli elementi della pagina

Combinatori

- **Discendente:** `div p` — ogni `<p>` dentro un `<div>`
- **Figlio diretto:** `ul > li` — solo `<li>` figli diretti di `<ul>`
- **Gruppo:** `h1, h2, h3` — stessa regola a più selettori

Pseudo-classi (esempi)

- `a:hover` — quando il mouse passa sopra un link
- `li:first-child` — il primo `<li>` di un elenco
- `input:focus` — campo di input attivo/selezionato

I **selettori** sono il meccanismo con cui il CSS individua gli elementi HTML a cui applicare uno stile. Quelli fondamentali si basano sul tipo di elemento, sulla classe o sull'identificativo.

CSS: selettori

Selettori fondamentali

- **Elemento:** `p { ... }` — seleziona tutti i `<p>`
- **Classe:** `.info { ... }` — elementi con `class="info"`
- **ID:** `#header { ... }` — elemento unico con `id="header"`
- **Universale:** `* { ... }` — tutti gli elementi della pagina

Combinatori

- **Discendente:** `div p` — ogni `<p>` dentro un `<div>`
- **Figlio diretto:** `ul > li` — solo `<li>` figli diretti di `<ul>`
- **Gruppo:** `h1, h2, h3` — stessa regola a più selettori

Pseudo-classi (esempi)

- `a:hover` — quando il mouse passa sopra un link
- `li:first-child` — il primo `<li>` di un elenco
- `input:focus` — campo di input attivo/selezionato

I **combinatori** CSS permettono di selezionare elementi in base alla loro **posizione nell'albero DOM** rispetto ad altri elementi, non per quello che sono ma per dove si trovano.

È come navigare l'albero DOM, ma lo fa il browser automaticamente al posto di JavaScript.

CSS: selettori

Selettori fondamentali

- **Elemento:** `p { ... }` — seleziona tutti i `<p>`
- **Classe:** `.info { ... }` — elementi con `class="info"`
- **ID:** `#header { ... }` — elemento unico con `id="header"`
- **Universale:** `* { ... }` — tutti gli elementi della pagina

Combinatori

- **Discendente:** `div p` — ogni `<p>` dentro un `<div>`
- **Figlio diretto:** `ul > li` — solo `<li>` figli diretti di `<ul>`
- **Gruppo:** `h1, h2, h3` — stessa regola a più selettori

Pseudo-classi (esempi)

- `a:hover` — quando il mouse passa sopra un link
- `li:first-child` — il primo `<li>` di un elenco
- `input:focus` — campo di input attivo/selezionato

Un elemento HTML è sempre lo stesso tag ma può trovarsi in stati diversi:

- un link può essere **già visitato** o **non ancora**
- un bottone può essere **premuto** in questo momento
- un campo di testo può essere **attivo** (l'utente sta scrivendo) o no
- un `<li>` può essere il **primo** o l'**ultimo** della lista
- Le pseudo-classi permettono di stilizzare questi stati **senza aggiungere classi nel codice HTML**.

CSS: proprietà visive e colori

```
<style type="text/css">
  h1{
    color: red;
  }
  h2{
    color: blue;
    font-style: italic;
  }
  p{
    color: white;
    background-color: black;
  }
</style>
```

```
<head>
  <link rel="stylesheet" type="text/css" href="theme.css">
</head>
```

Blocco <style>

Tre regole con **selettori di elemento**:

- *h1* → testo rosso (*color*)
- *h2* → testo blu + corsivo (*color* + *font-style*)
- *p* → testo bianco su sfondo nero (*color* + *background-color*)
- Lo stile è inserito direttamente nel documento HTML, dentro <style>

Tag <link>

Collega un file *theme.css* esterno tramite <link> nel <head>

- Attributi chiave:
 - *rel*="stylesheet" → tipo di risorsa
 - *href*="theme.css" → percorso del file
- → *Metodo preferito: separa contenuto e stile!*

CSS: proprietà visive e colori

```
<style type="text/css">
  h1{
    color: red;
  }
  h2{
    color: blue;
    font-style: italic;
  }
  p{
    color: white;
    background-color: black;
  }
</style>
```

```
<head>
  <link rel="stylesheet" type="text/css" href="theme.css">
</head>
```

Esempio da: <http://www.slideshare.net/tpryan/introduction-to-html5-11711719>



whichElement?

Trying to answer that age old question:

Should that be a div, a span, or something else?

- Home
- Contribute
- About

One of the main challenges we see in building semantic content is picking what tag to use when. This site seeks to help with that. Now, before we get all judgy and preachy let me get a few tenants out there:

Non è detto che sia un bello stile!


```
<style type="text/css">
  h1{
    color: red;
    font-size: 16px;
  }
  h2{
    color: blue;
    font-size: 26px;
  }
  ul{
    list-style: none;
    clear: both;
    display: block;
    width: 100%;
    height: 30px;
    padding: 0;
  }
  li{
    float: left;
    padding-right: 20px;
  }
</style>
```

CSS: proprietà visive e colori

Regole su h1 e h2

- *h1* → rosso, 16px
- *h2* → blu, 26px
- Usano *color* e *font-size* (qui in **px**, unità assoluta)

Regola su *ul* (menu orizzontale)

- *list-style*: none → rimuove i pallini
- *display*: block → contenitore a blocco
- *width*: 100% → occupa tutta la larghezza
- *clear*: both → annulla float precedenti
- *height*: 30px;
- *padding*: 0 → altezza fissa, nessun padding

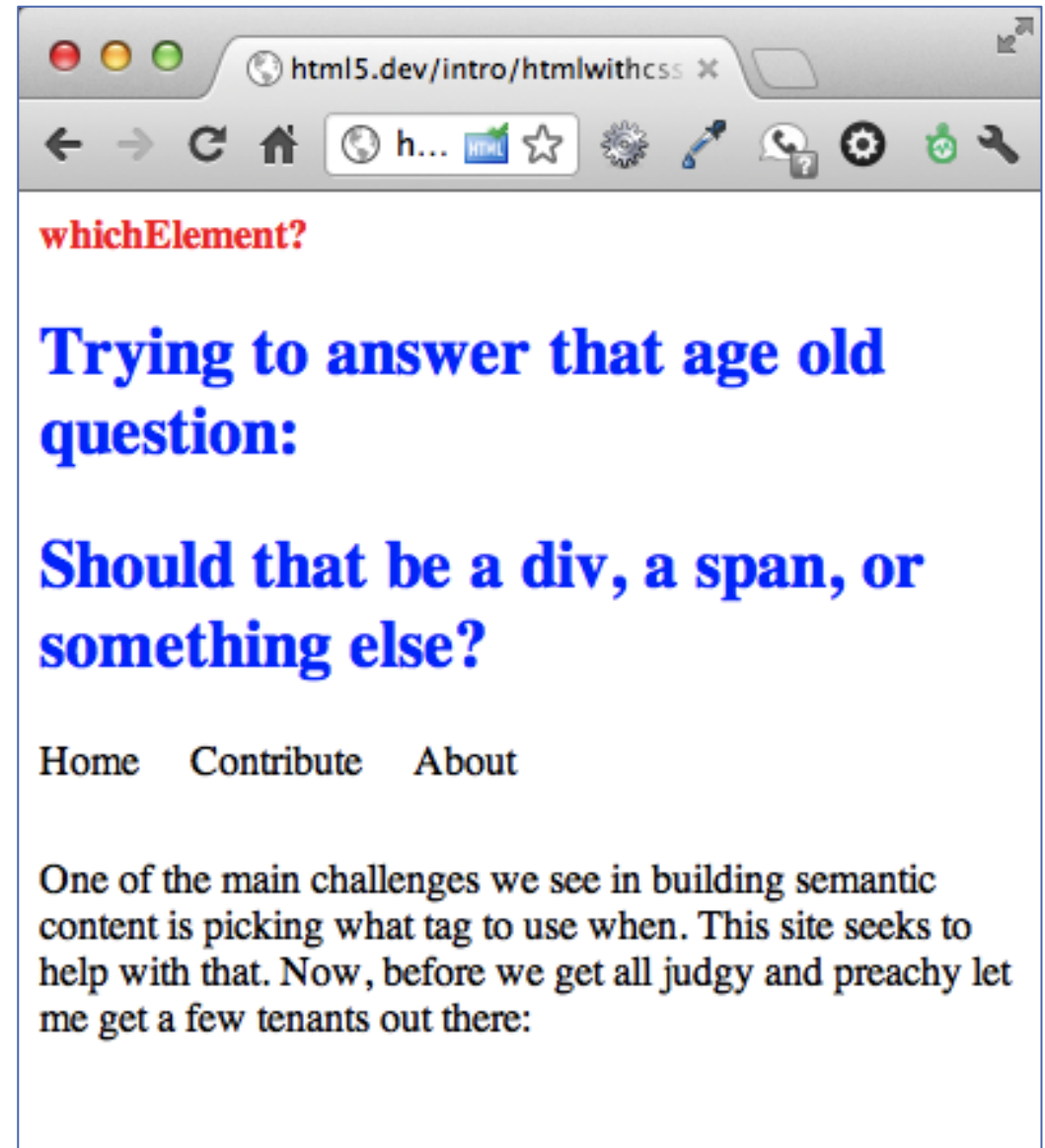
Regola su *li* (voci affiancate)

- *float*: left → dispone le voci in orizzontale
- *padding-right*: 20px → spaziatura tra le voci
- → *Pattern classico per barre di navigazione orizzontali*

Esempio da: <http://www.slideshare.net/tpryan/introduction-to-html5-11711719>

```
<style type="text/css">
  h1{
    color: red;
    font-size: 16px;
  }
  h2{
    color: blue;
    font-size: 26px;
  }
  ul{
    list-style: none;
    clear: both;
    display: block;
    width: 100%;
    height: 30px;
    padding: 0;
  }
  li{
    float: left;
    padding-right: 20px;
  }
</style>
```

CSS: proprietà visive e colori



Qualche dettaglio in più sui CSS: media query

Le media query sono costrutti CSS che permettono di **applicare regole di stile in modo condizionale**, in base alle **caratteristiche del dispositivo o del contesto di accesso**

Si possono controllare ad esempio:

- Larghezza e altezza (width, height) del device o della finestra
- Orientamento dello schermo (landscape/portrait)
- Risoluzione

Se la pagina è più larga di 480 pixel (e si sta visualizzando sullo schermo), applica determinati stili agli elementi con id “leftsidebar” (un menu) e “main” (la colonna centrale)

```
@media screen and (min-width: 480px){  
  #leftsidebar {width: 200px; float: left;}  
  #main {margin-left: 216px;}  
}
```

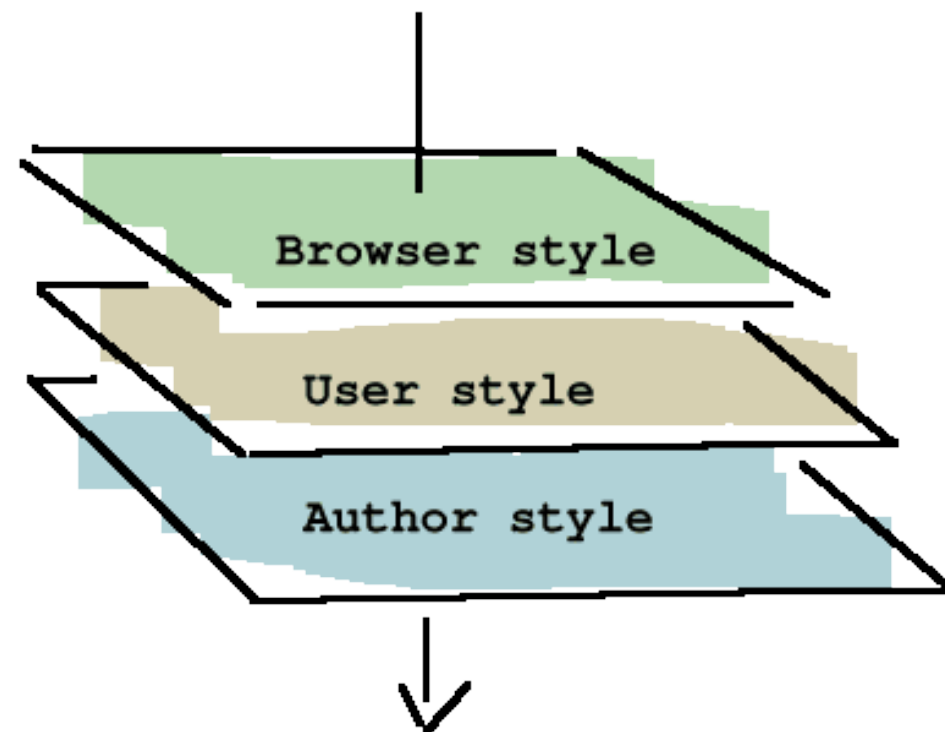
Curiosità: perché Cascading in CSS?

Esistono **potenzialmente diversi stylesheet**:

- L'**autore** della pagina in genere ne specifica uno (il modo più comunemente inteso) o **più d'uno**
- Il **browser** ne ha uno, o un vero e proprio CSS o simulato nel loro codice
- Il **lettore**, l'utente del browser, ne può definire uno proprio per customizzare la propria esperienza

Dei **conflitti** sono quindi possibili (inevitabili), ed è necessario definire un algoritmo per decidere quale stile vada applicato a un elemento

- Non entriamo nel dettaglio, ma la priorità tiene conto, in ordine di importanza, dei seguenti fattori associati alle regole:
 - Importanza (flag specifico !important per un attributo)
 - Specificità (per esempio, id > class > tag)
 - Ordine nel sorgente (lo più "recente" vince)



https://outer-court.com/tech/web_development/css.htm

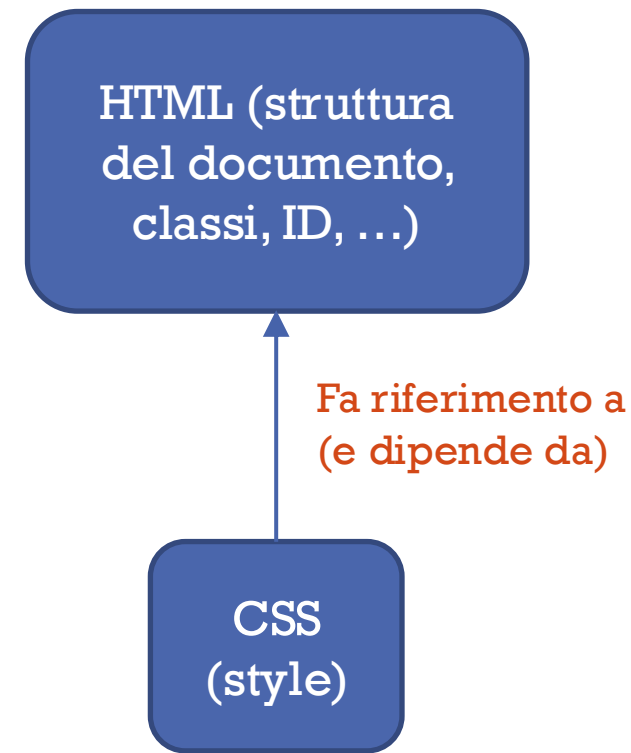
Qualche considerazione

L'impostazione di HTML e CSS **separa nettamente** il contenuto dalla modalità di **visualizzazione**

- Ricordo che quello di “separation of concerns” è un principio di design altamente desiderabile in contesto informatico
- Purtroppo lo stile invece può avere un **forte coupling** con il contenuto

```
div > ul > li:first-child a { color: red; }
```

Questo stile dipende dalla **struttura esatta** dell'HTML: div che contiene ul che contiene li.
Se domani ristrutturiamo l'HTML, il CSS **smette di funzionare**



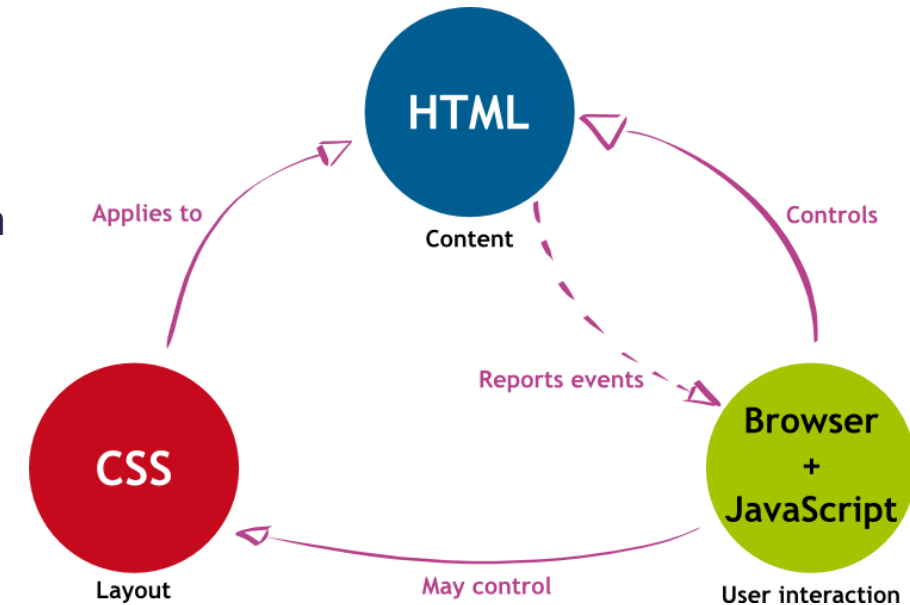
Guardando avanti: JavaScript

Abbiamo capito cosa sono HTML, CSS e DOM...

- ... che ci forniscono il necessario per definire contenuti (web), loro struttura (comprese regole di accesso e modifica), stile grafico

Manca il modo di definire e governare la dinamicità della pagina, il modo della pagina di cambiare in modo reattivo al comportamento dell'utente o anche proattivo, senza necessariamente dover attendere uno stimolo dall'utente

Javascript è il linguaggio di scripting che svolge questa funzione.



<https://1land4tht.wordpress.com/2013/11/21/cascading-style-sheets/>

XML = eXtensible Markup Language

XML si è imposto come standard de facto per lo scambio di *informazioni semi-strutturate*

Inizialmente pensato come successore di HTML come linguaggio per il Web, è diventato un linguaggio generico di cui HTML5 è una versione specifica

Caratteristiche dei documenti XML

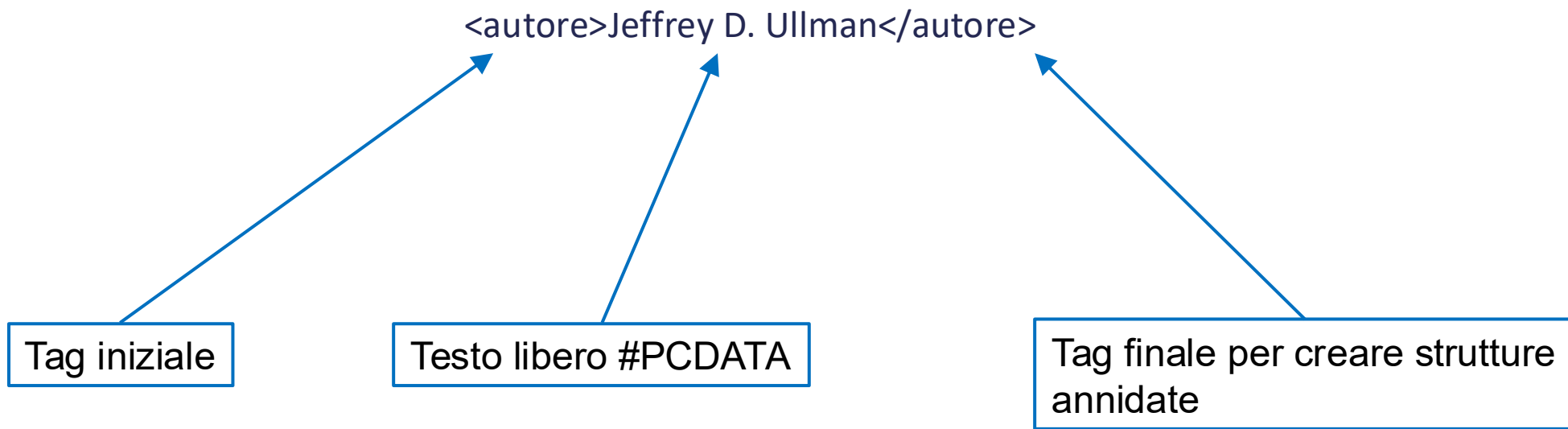
- Marcatura *descrittiva* e non procedurale (descrive la struttura logica)
- Marcatura definisce la *struttura logica* del documento
- Flessibilità: le etichette possono cambiare in base all'applicazione
- Il *tipo di documento* specifica la struttura della marcatura ammessa
 - *Document Type Declaration* o *DTD* (è parte dello standard XML)

Markup language

Markup: tutto ciò che non è testo del documento (dalla specifica W3C)

- Tag di apertura / chiusura: `<tag>` e `</tag>`
- Tag vuoto (empty-element): `<br/>`
- Riferimenti a entità, commenti, sezioni CDATA, dichiarazioni, istruzioni di processing

Character data: tutto il testo che *non* è markup — il contenuto testuale vero e proprio del documento



Documenti XML

Documento XML: oggetto testuale che rispetta la specifica XML (dalla specifica W3C)

- **Well-formed:** rispetta le regole sintattiche di XML
- **Valid:** rispetta inoltre i vincoli di un DTD o XML Schema

Condizioni per essere well-formed

- Rispetta la grammatica XML (produzione *document*)*
- Soddisfa tutti i vincoli di well-formedness*
- Ogni entità referenziata (direttamente o indirettamente) è a sua volta well-formed
- Contiene almeno un elemento

Elemento radice (root): esiste esattamente un elemento che contiene tutti gli altri; nessuna sua parte appare come contenuto di un altro elemento

* <https://www.w3.org/TR/xml/>

Well-formed document

Un documento che rispetta la specifica di XML è detto *well-formed*:

- costituito da un *insieme di elementi annidati*
- un *elemento* è costituito da una coppia di etichette di apertura e di chiusura e da tutto quello che racchiudono

```
<?xml version="1.0"?>
<media>
  <cds>
    <cd quantity="10">
      <title>Glory and Consequence</title>
      <artist>Ben Harper</artist>
      <album>The Will to Live</album>
      <album>Hello</album>
    </cd>
    <cd quantity="6">
      <title>Bigmouth Strikes Again</title>
      <artist>The Smiths</artist>
      <album>Meat Is Murder</album>
    </cd>
  </cds>
  <books>
    <book quantity="3">
      <title>Cave Birds</title>
      <author>Ted Hughes</author>
    </book>
  </books>
</media>
```

DTD – Document Type Declaration

Document Type Definition (DTD): grammatica che definisce le strutture ammesse per una classe di documenti XML (dalla specifica W3C)

Può essere *interna* (inline nel documento) o *esterna* (file .dtd separato); la DTD finale è l'unione dei due subset

Una DTD contiene dichiarazioni (markup declarations) di

- *Tipi di elemento*: quali elementi esistono e come si annidano
- *Attributi*: quali attributi sono ammessi per ogni elemento e di che tipo
- *Entità e notazioni*: abbreviazioni e formati esterni

Documenti validi

Un documento XML *ha* una *struttura arbitraria*:

- elementi con nome arbitrario
- attributi arbitrari in qualunque elemento
- elementi organizzati in modo arbitrario

Un *DTD* specifica quali sono le strutture ammesse:

- specifica il *tipo del documento*
- contiene *dichiarazioni*:
 - di *tipi di elemento*
 - di *attributi*
 - di *entità*

```
<?xml version="1.0"?>
<!DOCTYPE Media SYSTEM "media.dtd">
<Media>

  . . .

</Media>
```

File media.dtd

```
<!ELEMENT media (cds | books)>
<!ELEMENT cds (cd*)>
<!ELEMENT cd (title, artist, album+)>
<!ATTLIST cd quantity CDATA #REQUIRED>
<!ELEMENT title (#PCDATA)>
<!ELEMENT artist (#PCDATA)>
<!ELEMENT album (#PCDATA)>
<!ELEMENT books (book*)>
<!ELEMENT book (title, author)>
<!ATTLIST book quantity CDATA #REQUIRED>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
```

Tipo testo libero

Limitazioni delle DTD

Le DTD descrivono solo la "grammatica" del file XML, non la struttura dettagliata e/o i tipi

Questa descrizione grammaticale ha alcune evidenti limitazioni:

- Non si può esprimere che un elemento **"length"** debba contenere un numero non negativo (*vincoli sul tipo del valore di un elemento o attributo*)
- L'elemento **"unit"** dovrebbe essere ammesso solo quando è presente **"amount"** (*vincoli di co-occorrenza*)
- L'elemento **"comment"** dovrebbe poter comparire ovunque (*flessibilità dello schema*)
- Non esiste il *subtyping* / ereditarietà (*riuso delle definizioni*)
- Non esistono *chiavi composite* (*integrità referenziale*)

XML Schema

- **XML Schema** è un linguaggio di schema più sofisticato che affronta i limiti delle DTD
- Supporta
 - *Tipizzazione dei valori*
 - Es. integer, string, ecc.
 - Inoltre, vincoli su valori min/max
 - *Tipi complessi* definiti dall'utente
 - Molte altre funzionalità, tra cui
 - *vincoli di unicità* e *chiave esterna*, *ereditarietà*
- XML Schema è a sua volta specificato in *sintassi XML*, a differenza delle DTD
 - Rappresentazione più standard, ma verbosa
- XML Schema è integrato con i *namespace*
- Attenzione: XML Schema è significativamente più complesso delle DTD.

DTD vs. struttura di uno Schema

■ DTD

```
<!ELEMENT e1 ((e2,e3?)+|e4)>
```

Definisce che l'elemento e1 può contenere

- o solo e4,
- o una sequenza ripetuta di e2 seguito opzionalmente da e3.

I simboli

- , (sequenza),
- ? (zero o uno),
- + (uno o più) e
- | (alternativa)

funzionano esattamente come nelle espressioni regolari, ma applicati a tag XML invece che a caratteri.

■ Schema

```
<element name="e1">  
  <complexType>  
    <choice>  
      <sequence maxOccurs="unbounded">  
        <element ref="e2"/>  
        <element ref="e3" minOccurs="0"/>  
      </sequence>  
      <element ref="e4"/>  
    </choice>  
  </complexType>  
</element>
```

DTD vs. struttura di uno Schema — esempio concreto

■ DTD

```
<!ELEMENT cd
  (title, artist, album+)>

<!ELEMENT title (#PCDATA)>
<!ELEMENT artist (#PCDATA)>
<!ELEMENT album (#PCDATA)>

// cd contiene: title, artist e
// almeno un album (+)
```

l'elemento **cd** è definito come

- una sequenza ordinata di **title**,
- **artist** e
- uno o più **album**,

quindi l'ordine è vincolato e album deve comparire almeno una volta.

■ Schema

```
<xs:element name="cd">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="title"/>
      <xs:element ref="artist"/>
      <xs:element ref="album" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

<xs:element name="title" type="xs:string"/>
<xs:element name="artist" type="xs:string"/>
<xs:element name="album" type="xs:string"/>
```



```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

```
  <xs:element name="media">
    <xs:complexType>
      <xs:choice>
        <xs:element ref="cds"/>
        <xs:element ref="books"/>
      </xs:choice>
    </xs:complexType>
  </xs:element>
```

```
  <xs:element name="cds">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="cd" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

```
  <xs:element name="cd">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="title"/>
        <xs:element ref="artist"/>
        <xs:element ref="album" maxOccurs="unbounded"/>
      </xs:sequence>
      <xs:attribute name="quantity" type="xs:integer" use="required"/>
    </xs:complexType>
```

```
    <xs:element name="title" type="xs:string"/>
    <xs:element name="artist" type="xs:string"/>
    <xs:element name="album" type="xs:string"/>
```

```
  <xs:element name="books">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="book" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

```
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="title"/>
        <xs:element ref="author"/>
      </xs:sequence>
      <xs:attribute name="quantity" type="xs:integer" use="required"/>
    </xs:complexType>
  </xs:element>
```

```
  <xs:element name="author" type="xs:string"/>
```

```
</xs:schema>
```

Tipo intero

OGGETTI JSON

JSON: caratteristiche e definizione

JSON (JavaScript Object Notation) è

- un *formato leggero* per lo scambio di dati
- facile da leggere e scrivere per gli esseri umani; facile da analizzare e generare per le macchine
- basato su un sottoinsieme dello standard del linguaggio di programmazione JavaScript ECMA-262 3a edizione - Dicembre 1999

JSON non è

- un formato di documento.
- un linguaggio di markup.
- un formato di serializzazione generale.
 - Nessuna struttura ciclica/ricorrente.
 - Nessuna struttura invisibile.
 - Nessuna funzione.

JSON: strutture dati fondamentali

JSON (JavaScript Object Notation) è

- un formato testuale completamente indipendente dal linguaggio
 - utilizza convenzioni familiari ai programmatori delle lingue della famiglia C, tra cui C, C++, C#, Java, JavaScript, Perl, Python e molti altri

JSON si basa su due strutture dati universali:

- **Una raccolta di coppie nome/valore.** In vari linguaggi, questo è realizzato come un *object*, record, struct, dizionario, tabella hash, lista con chiave o array associativo
- **Un elenco ordinato di valori.** Nella maggior parte dei linguaggi, questo è realizzato come un *array*, vettore, lista o sequenza

Praticamente tutti i linguaggi di programmazione moderni li supportano

JSON: sintassi e tipi di base

Regole di sintassi JSON

- Il dato minimo è una *coppia nome/valore*
- I dati sono separati da virgole
- Le parentesi graffe contengono *oggetti*
- Le parentesi quadre contengono *array*

I tipi di base sono

- **Number** intero, reale o a virgola mobile
- **String** Unicode tra virgolette doppie con barre rovesciate
- **Boolean** true e false
- **Array** sequenza ordinata di valori separati da virgole racchiusi in parentesi quadre
["Sunday", "Monday", "Tuesday", "Wednesday"]
- **Object** raccolta di coppie "key":value coppie separate da virgole racchiuse in parentesi graffe
{ "name": "Jack (\\"Bee\\" Nimble", "format": { "type": "rect", "width": 120, "interlace": false}}
- null

JSON: oggetti, array e note pratiche

Array vs *Oggetti*

- Usare gli array quando i nomi delle chiavi sono interi sequenziali.
- Usare gli oggetti quando i nomi delle chiavi sono stringhe arbitrarie.
- Gli oggetti sono implementati dal modello *Associative Array*.

Varie:

- MIME Media Type: **application/json**
- Codifica dei caratteri
 - Esclusivamente UNICODE.
 - Predefinito: **UTF-8**.
 - UTF-16 e UTF-32 sono consentiti.
- Senza versione
 - JSON non ha un numero di versione.
 - Non sono previste revisioni alla grammatica JSON: JSON è molto stabile.

JSON: come utilizzarlo

JSON assomiglia ai dati

- I valori semplici di JSON sono gli stessi utilizzati nei linguaggi di programmazione.
- Le strutture di JSON assomigliano alle strutture convenzionali dei linguaggi di programmazione.
 - L'oggetto di JSON è *record, struct, object, dizionario, hash, array associativo...*
 - L'array di JSON è *array, vettore, sequenza, lista...*

Regole

- Un *decoder* JSON deve accettare tutto il testo JSON ben formato.
- Un decoder JSON può anche accettare testo non-JSON.
- Un *encoder* JSON deve produrre solo testo JSON ben formato.
- Sii conservativo in ciò che fai, sii liberale in ciò che accetti dagli altri.

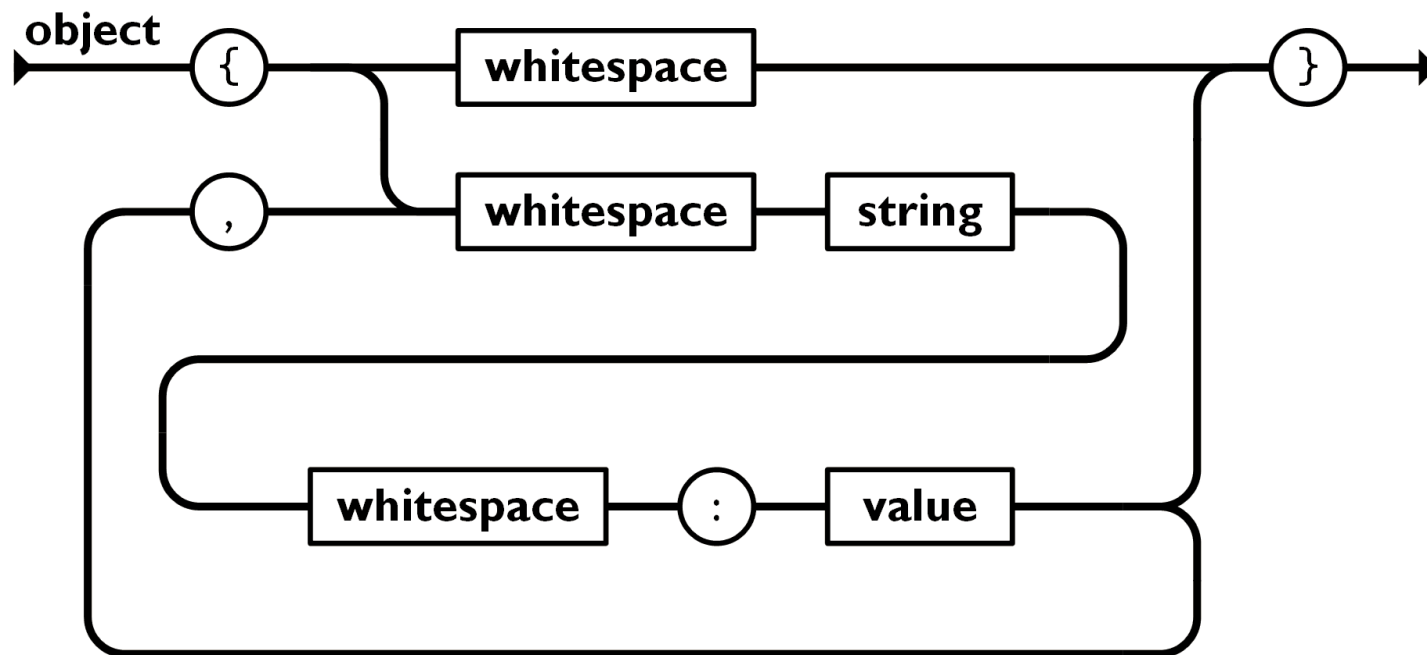
Oggetti JSON

Un *object* è un insieme non ordinato di coppie nome/valore

Un oggetto inizia con *{parentesi graffa sinistra* e termina con *}parentesi graffa destra*

Ogni nome è seguito da *:due punti*

Le coppie nome/valore sono separate da *,virgola*



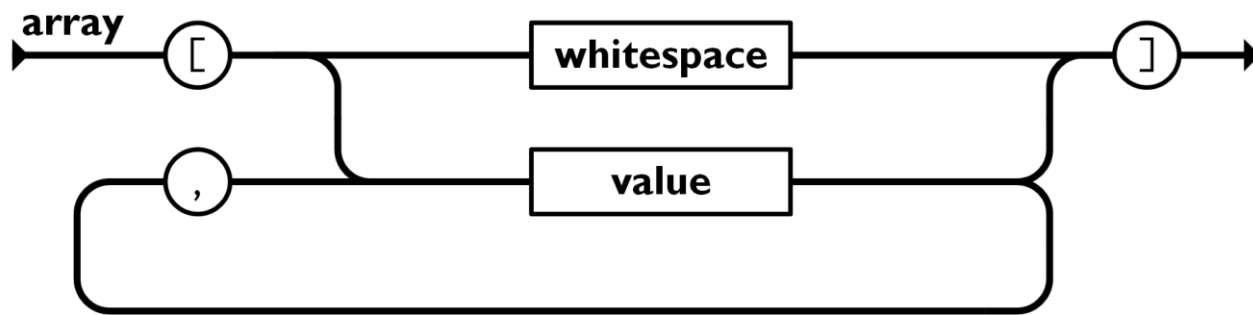
```
{  
  "nome": "Mario Rossi",  
  "eta": 30,  
  "attivo": true  
}
```


Array JSON

Un *array* è una raccolta ordinata di valori

Un array inizia con *[parentesi quadra sinistra* e termina con *]parentesi quadra destra*

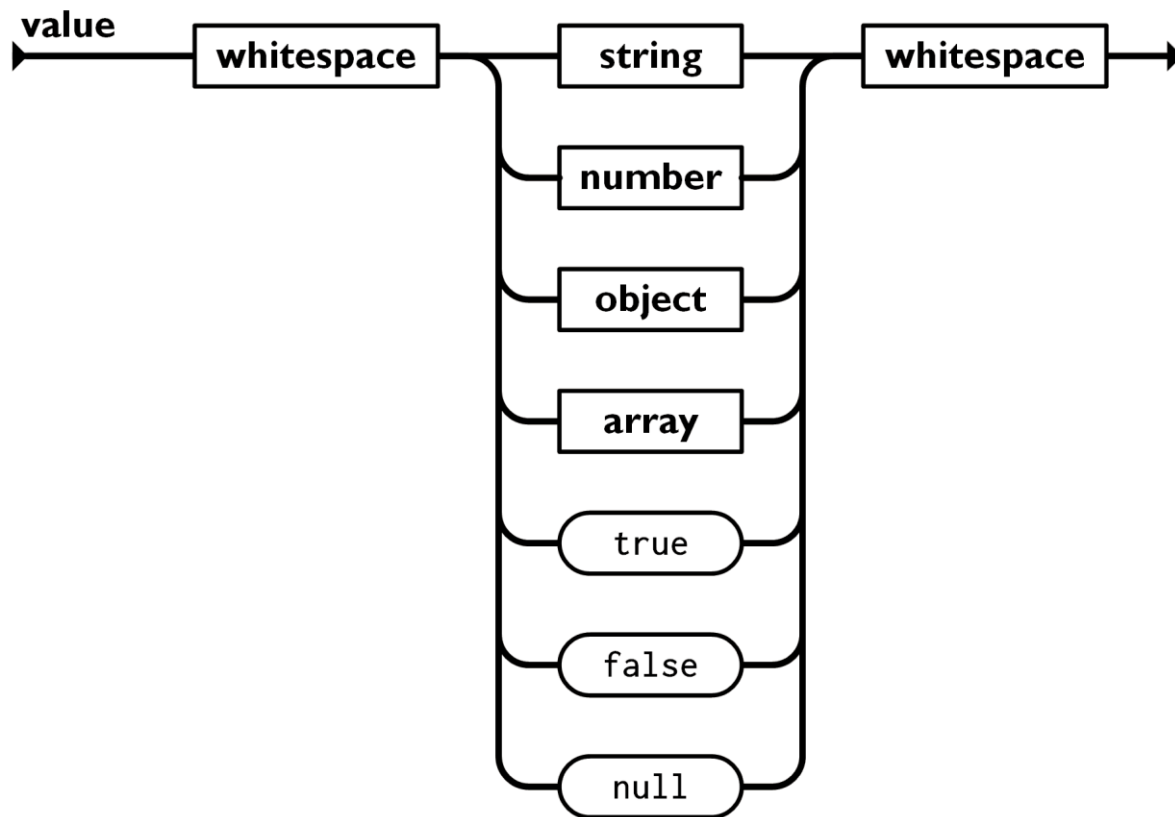
I valori sono separati da *,virgola*



```
[  
  {  
    "nome": "Mario Rossi",  
    "eta": 30  
  },  
  {  
    "nome": "Anna Bianchi",  
    "eta": 25  
  }  
]
```

Valore JSON

Un *value* può essere una *string* tra virgolette doppie, oppure un *number*, o *true* o *false* o *null*, o un *object* o un *array*.
object e *array* le strutture possono essere annidate.



```
{  
  "nome": "Mario",  
  "eta": 30,  
  "attivo": true,  
  "eliminato": false,  
  "note": null,  
  "indirizzo": {  
    "citta": "Milano"  
  },  
  "tag": ["web", "json"]  
}
```

JSON Schema

Cos'è

- Un vocabolario JSON per descrivere e validare la struttura di altri documenti JSON.

A cosa serve

- Validare dati in ingresso ad API e form.
- Documentare contratti tra sistemi.
- Generare interfacce e codice automaticamente.

Analogia

- JSON Schema sta a JSON come XSD sta a XML.

Schema per un oggetto persona

Proprietà obbligatorie:

nome → string

eta → number, minimo 0

Proprietà opzionale:

attivo → boolean

Validazione: questo oggetto è valido?

{ "nome": "Mario", "eta": 30 } ✓

{ "nome": "Anna" } ✗ (manca eta)

{ "nome": "Luca", "eta": -5 } ✗ (eta < 0)

```
{
  "$schema": "http://json-schema.org/draft-07/schema",
  "type": "object",
  "properties": {
    "nome": { "type": "string" },
    "eta": { "type": "number", "minimum": 0 },
    "attivo": { "type": "boolean" }
  },
  "required": ["nome", "eta"]
}
```